



FOR JAVA DEVELOPERS SPLITTING AN APP INTO MODULES

It Exists. Still Refused.

Spring Boot to NestJS, episode 3

A provider can exist, and still be private. Registered is not visible.

In a typical Spring Boot application with one application context, a component that scanning discovers can be injected anywhere in that context. Packages organise code; they do not create a boundary for dependency injection.

NestJS is different. A provider belongs to exactly one module and is private to it by default. The NestJS documentation says it directly: a module encapsulates its providers.

The versions on this sheet were current on 23 September 2026: NestJS 12 and Spring Boot 4.1.1. The exact wording of the injection errors and the behaviour of transient scope are release details, so check the date on anything else you read about them.

In scope

- Why a provider Nest built can still be refused one module away
- The two doors: exports on the owning module, imports on the consumer
- Telling two almost identical injection errors apart, and the limit of that
- Factory providers, and the token as the lookup identity
- Default, request and transient scope against Spring singleton and prototype
- Global modules, and the documentation's own warning about them

Out of scope

- Lifecycle hooks, beyond one line: they belong with startup and shutdown
- Dynamic modules and forRoot, which have their own episode
- Async and the event loop, which is next

One service, two modules, every state measured. Everything here is reproducible with a single script.

01 What the container actually said

PaymentsModule on its own	<code>isPaid('pay_1') → true</code>
Interface as the parameter type	<code>refused, no name: "the argument at index [0]"</code>
Class, neither door open	<code>refused, named: "PaymentsService at index [0]"</code>
imports only	<code>the same message</code>
exports only	<code>the same message</code>
exports and imports	<code>receipt for pay_1</code>
Transient, one consumer injecting it twice	<code>one instance: 3 and 3</code>
Spring prototype, the same shape	<code>two instances: 3 and 4</code>

Every row came off a terminal on 23 September 2026. Reproduce all of it with `scripts/verify-modules.sh`.

03 It exists, and it is still refused

The payment service below is correct and Nest builds it. Start its module alone and it answers. Then a receipts service in its own module asks for it through the constructor, and the container refuses.

Nothing is missing. The provider is registered in the payments module and is simply not visible from the receipts module.

payments.service.ts and payments.module.ts

```
@Injectable()
export class PaymentsService {
  private readonly paid = new Set<string>(['pay_1']);
  isPaid(id: string) {
    return this.paid.has(id);
  }
}

@Module({ providers: [PaymentsService] })
export class PaymentsModule {}
```

what the container prints

```
Nest can't resolve dependencies of the ReceiptsService (?).
Please make sure that the argument PaymentsService at index [0]
is available in the ReceiptsModule module.
```

A module encapsulates its providers by default; only exported providers of an imported module can be injected.

NestJS documentation, Modules, Custom providers and Injection scopes, checked 2026-09-23.

The provider resolves on its own and is refused from another module until both doors are open.

Run on a real machine, 2026-09-23, by scripts/verify-modules.sh in the course repository.

04 Open both doors

Importing a module gives you only what that module exports. So importing alone produces exactly the same refusal, and so does exporting alone. Both lines are needed.

```
the owning module exports it

@Module({
  providers: [PaymentsService],
  exports: [PaymentsService],
})
export class PaymentsModule {}
```

```
the consumer imports that module

@Module({
  imports: [PaymentsModule],
  providers: [ReceiptsService],
})
export class ReceiptsModule {}

Measured result: receipt for pay_1.
```

imports only, exports only, and neither all print the identical message.

Run on a real machine, 2026-09-23, by scripts/verify-modules.sh in the course repository.

05 Two errors that look the same

When no name appears where the dependency should be, the parameter type was erased, usually because it was an interface: give it a token.

When the class is named, Nest knows what you asked for, and the problem is visibility: check the doors. The message does not say which door is shut.

```
no name: the type was erased

...the argument at index [0] is available in the current module.
```

```
a name: check exports and imports

...the argument PaymentsService at index [0] is available in the ReceiptsModule module.
```

Both messages as printed by NestJS 12; the distinction holds in the cases measured, not as a universal decoder.

Run on a real machine, 2026-09-23, by scripts/verify-modules.sh in the course repository.

06 The token is the identity

Two gateways built from one class, each by a factory reading the same configuration. A factory provider is the nearest thing Nest has to a bean method, and because each is registered under its own token, the consumer asks for each one by token. In Spring you would reach for a qualifier.

two providers, one class

```
{
  provide: PRIMARY_GATEWAY,
  useFactory: (cfg: { host: string; timeoutMs: number }) =>
    new Gateway(`https://${cfg.host}/v2`, cfg.timeoutMs),
  inject: [CONFIG],
},
{
  provide: FALLBACK_GATEWAY,
  useFactory: (cfg: { host: string; timeoutMs: number }) =>
    new Gateway(`https://${cfg.host}/v1`, cfg.timeoutMs * 2),
  inject: [CONFIG],
},
```

Measured: primary v2 (2000ms), fallback v1 (4000ms).

A factory provider supplies an instance from a function, with dependencies listed in inject.

NestJS documentation, Modules, Custom providers and Injection scopes, checked 2026-09-23.

07 Transient is not quite prototype

The default matches Spring: one instance for the application. Request scope creates one per request, and the documentation warns it slows responses and spreads to everything that depends on it.

Transient sounds like prototype and mostly behaves like it. The difference: one consumer injecting it twice gets the same instance in Nest and two in Spring. Transient is one per consumer; prototype is one per injection point. And `get()` refuses a transient provider; use `resolve()`.

counted on both stacks

	Nest transient	Spring prototype
two different consumers	1 and 2	1 and 2
one consumer, injected twice	3 and 3	3 and 4

Transient providers are not shared across consumers; each consumer receives a new, dedicated instance.

NestJS documentation, Modules, Custom providers and Injection scopes, checked 2026-09-23.

A request-scoped dependency makes its consumers request-scoped too.

NestJS documentation, Modules, Custom providers and Injection scopes, checked 2026-09-23.

The Spring singleton is per container and per bean, not the Gang of Four singleton.

Spring Framework documentation, Bean scopes, checked 2026-09-22.

Before you split a Nest app into modules

1. Export every provider another module needs, from the module that owns it
2. Import the owning module into the consumer, not the provider class
3. Read the injection error for a name before opening any file
4. Give an interface parameter a token; there is no type left to match at runtime
5. Register two instances of one class under two tokens, not two classes
6. Reach for `Global()` only for genuinely cross-cutting modules such as configuration



Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

Facebook

<https://www.facebook.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>

Buy me sadza

<https://buymeacoffee.com/codewithsam>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Node.js is a trademark of the OpenJS Foundation. Java is a trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. NestJS is the property of its owner. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them, and no third party artwork is reproduced here.

Technical claims verified 2026-09-23 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.